TESTING FRAMEWORK IMPLEMENTATION

The project incorporates four independent unit testing suites to ensure comprehensive validation of functionality, reliability, and code quality across different components of the system.

1. PYTEST TEST SUITE

Target Component: SQL Quality Linter and Rule Parser Engine.

Testing Scope: The Pytest suite validates the core SQL linting functionality, including detection of SELECT * violations (RULE-001), camelCase identifier formatting violations (RULE-002), and verification of valid SQL queries.

Methodology: Standard Python assertion-based testing is used to evaluate rule compliance and ensure accurate linting behavior.

File Location: tests/test_sql_linter.py

2. XUNIT TEST SUITE

Target Component: XML Schema Generation and Continuous Integration Reporting.

Testing Scope: The XUnit test suite verifies schema generation accuracy, report compilation, execution consistency, and validation of generated outputs.

Methodology: XUnit testing structures are used to compare expected schema outputs, execution results, failure messages, and runtime behavior.

File Location: src/xunit.test.ts

3. VITEST TEST SUITE

Target Component: Core Server Functions and Validation Utilities.

Testing Scope: The Vitest suite validates cryptographic helper functions, database snake_case identifier enforcement rules, and SSO JWT token validation mechanisms.

Methodology: Testing is performed using describe(), expect(), and it() hooks within the Vite and Node.js environment.

File Location: src/server.test.ts

4. NODE.JS NATIVE TEST RUNNER

Target Component: Administrative Permission Controls and Performance Constraints.

Testing Scope: This test suite verifies administrator permission mappings, access control rules, and Q-Score performance metric thresholds.

Methodology: Testing is performed using the native Node.js testing framework, including node:test and node:assert libraries, without relying on external testing frameworks.

File Location: src/native_node.test.js

DYNAMIC BACKEND TEST EXECUTION

To streamline execution and reporting, the backend server runner located in server.ts has been configured to execute all four test suites independently and consolidate their outputs into a unified execution log.

Pytest: python3 -m pytest tests/test_sql_linter.py -v

XUnit: npx vitest run src/xunit.test.ts

Vitest: npx vitest run src/server.test.ts

Node.js Native Runner: node src/native_node.test.js

This architecture ensures comprehensive test coverage while maintaining separation between testing methodologies and execution environments.